responds asynchronously. Inside the `Source.futureSource`, you call the client's `follow` method, which is generated by Akka gRPC and returns a `Source[Location, NotUsed]`. Finally, you wrap this call in a future (an asynchronous call), and with it, you build the `Source.futureSource`.

Let's close this chapter with another common use case: adding elements to the source of a stream after the stream has been started.

16.4 Adding elements dynamically to a stream

Some streams do not have a source completely defined at the beginning. Instead, elements can be added dynamically. You can get a reference to the stream and pass it around so different parts of your program add new elements.

For example, suppose you have an initial set of elements that you want to process to create an inventory (a list of products). These initial items are fixed when the program starts, but you want to be able to add more later. To do so, you can use a `Source.queue`.

The API consists of two basic elements: the queue itself, `Source.queue`, which creates the `SourceQueue` that you can then pass around; and the materialization, `toMat`, to extract the `SourceQueue` object from the `Source.queue` method. Once you have the queue, you can add elements to it using `queue.offer(elem: T)`. This is shown in a test in the following listing.